

No Code Development (NCD): A Paradigm Shift Beyond “Vibe Coding”

Abstract

Abstract: This white paper proposes **No Code Development (NCD)** as a more accurate and professional term for the emerging paradigm often referred to as “vibe coding.” Coined informally to describe AI-assisted software creation through natural language, *vibe coding* has gained attention but mischaracterizes the discipline and rigor of the practice ¹ ². Drawing on Dinis Cruz’s firsthand experience – notably his preparation for the ThreatModCon 2025 keynote – we illustrate how NCD enables developers to rapidly build real software (apps, UIs, data visualizations) **without writing manual code**, using iterative prompts, AI tooling, and visual interfaces. We argue that “No Code Development” better encapsulates this workflow’s essence: a highly-iterative, feedback-driven process where the human developer focuses on intent and orchestration while the AI handles code generation. The paper examines why the term “vibe coding” falls short, highlights the productivity gains of NCD (as evidenced by the ThreatModCon case study), and discusses the “air gap” between NCD and traditional engineering – i.e. how context-switching into code disrupts creative flow ³. We further explore the irony that seasoned software engineers, far from being obsolete, are often the most effective NCD practitioners due to their domain knowledge and prompting skills ¹. Finally, we delineate why NCD deserves a distinct identity apart from conventional *low-code/no-code* platforms, particularly for technical teams seeking to leverage AI-driven development.

Introduction

In recent years, advances in generative AI – especially large language models (LLMs) like GPT-4, Codex, and others – have ushered in a new style of software creation. Instead of writing syntax line-by-line, developers can now “**program**” using natural language. This approach, sometimes nicknamed “**vibe coding**,” involves expressing the desired functionality or interface in plain English (or another human language) and letting an AI assistant translate those intentions into working code ². The goal is an AI-powered development environment where the human stays in the creative flow and the machine handles the tedious coding details ².

While the *concept* has proven powerful, the term “*vibe coding*” is increasingly seen as an **insufficient and misleading label** for this practice. The phrase originated casually (attributed to Andrej Karpathy in early 2025) and captured the imagination by suggesting one could “just vibe” and let code happen ⁴. However, *vibe coding* as a name fails to convey the true nature of the process. It implies an almost mystical, carefree activity – potentially trivializing the skill involved – and does not communicate that real development work is occurring. In professional and executive settings, describing an engineering process as a “vibe” can sound unserious or opaque. As Dinis Cruz observed, “*Vibe coding*” is a bad name for a good idea ¹. The practice in question is not about coding by “vibe” or guesswork; it is about leveraging AI to build software via high-level instructions. For clarity and credibility, we propose the term **No Code Development (NCD)** to describe this paradigm. NCD succinctly highlights that the developer writes **no code manually**, yet software **development** is indeed happening – the architecture, logic, and interfaces are being constructed through AI-driven means.

This paper, authored from the perspective of Dinis Cruz, delves into why **“No Code Development” (NCD)** better captures this new paradigm and how it addresses the shortcomings of “vibe coding” as a descriptor. We will use Dinis’s recent experiences – notably his ThreatModCon 2025 keynote preparations and subsequent reflections – as a case study to demonstrate the power and practicality of NCD. Key facets we explore include:

- Why *“vibe coding”* as a label can misrepresent the methodology and its seriousness.
- How **No Code Development** works and its core properties: no manual coding, iterative real-time feedback loops, and development via natural language + AI tools.
- The significant productivity benefits witnessed (e.g. rapidly creating multiple visualizations and tools for a live demo).
- The **“air gap”** between NCD and traditional coding – how having to shift into writing or reading code can break the creative flow ³.
- The irony that far from eliminating the need for skilled programmers, NCD often **amplifies** the need – with experienced developers excelling at NCD due to their knowledge in shaping prompts and verifying outputs ¹.
- Why NCD should be recognized as a distinct approach, separate from legacy *low-code/no-code* platforms, especially in contexts where technical teams are involved.

Structured as a white paper, the document provides an Abstract, Introduction, a detailed case study, discussions on core concepts and contrasts with traditional engineering, and sections on developer empowerment and strategic implications. We conclude with future directions for adopting NCD as a mainstream practice. Throughout, we cite Dinis Cruz’s own documentation and public communications to ground the discussion in real-world observations and evidence.

Case Study: ThreatModCon 2025 – Accelerating Development Without Coding

To illustrate the impact of No Code Development, we turn to a recent case study: **ThreatModCon 2025** in Barcelona, where Dinis Cruz delivered the closing keynote. The presentation – titled *“Scaling Supply Chain Security using Threat Modeling Semantic Knowledge Graphs and Maps”* – was not just notable for its content, but for *how* the content was prepared. In the lead-up to the conference, Cruz needed a series of custom **interactive visualizations and UI tools** to demonstrate concepts like knowledge graphs and threat maps in real time. Traditionally, creating even a few interactive prototypes or data-driven visuals for a talk could take days or weeks of coding effort. Instead, using the NCD approach (what was informally called *“vibe coding”* at the time), he was able to **iteratively “prompt” these UIs into existence without writing code by hand**.

Crucially, this was done with remarkable speed and flexibility. Cruz has noted that he was *“super happy with the UIs [he] created for this presentation,”* having **“vibe coded” them up to 5 minutes before the talk** ⁵. In other words, final adjustments and even new interface elements were being generated on the fly, right before he went on stage. This anecdote underscores a key benefit of NCD: the ability to **rapidly experiment and refine in real-time**. Because the workflow was driven by natural-language prompts and AI-generated code, Cruz could implement changes or try new ideas in seconds, then immediately see the results in a running application. This highly interactive, feedback-centric process allowed him to **visualize the ideas in his head** quickly and accurately ⁵. As he reflected afterward, NCD finally enabled him to explore and present concepts “in ways that truly visualize what I have in my head” – something traditional coding might not have allowed under tight time constraints ⁵.

Several concrete outcomes emerged from the ThreatModCon preparation that demonstrate the productivity of NCD:

- **Multiple Custom Tools in Days (or Hours):** Instead of hand-coding one or two demos, Cruz was able to create **multiple working visualizations and tools** for the keynote. Each was tailored to a specific concept (for example, a semantic graph explorer, a threat-map UI, etc.), effectively acting as mini software products developed almost in parallel. This was possible because generating a new tool was as straightforward as describing the desired functionality or interface to an AI assistant and iterating on the result.
- **Last-Minute Iteration:** Because no manual coding was required, making a change didn't risk breaking a fragile codebase at the eleventh hour. Cruz could confidently tweak the UI/UX "5 minutes before the talk" ⁵ – something few developers would dare to do with traditionally coded software right before a live demo. The AI-assisted development environment provided a safety net of sorts: if a prompt-led change didn't work as expected, one could simply undo or rephrase the request, all without digging into low-level code. This encourages a fluid, **fearless iteration cycle** that is hard to achieve in classic development.
- **Focus on Ideas, Not Syntax:** During the preparation, Cruz remained focused on the *ideas* and *messaging* of his talk – the actual content he wanted to convey – rather than being bogged down by writing boilerplate code or fixing routine bugs. In his LinkedIn summary of the experience, he indicated that by "vibe coding" the presentation UIs, he could concentrate on exploring the concepts around graphs and maps, and how best to visualize them, instead of wrestling with technical implementation details ⁵. This focus on a higher level of abstraction is a hallmark of NCD, and it paid off in the clarity and quality of the demos.

The success of the ThreatModCon demos served as a compelling proof-of-concept for No Code Development. It highlighted how an experienced developer could leverage AI tools to **dramatically compress the development timeline** without sacrificing quality or customization. The applications created were not canned demos or slides – they were genuine software artifacts (with real data and interactive elements) built in a fraction of the usual time. Perhaps most tellingly, this was achieved in a high-stakes setting (a live conference) which demonstrates the confidence NCD can instill when one has mastered the workflow. What once might have been considered an experimental "vibe" approach has matured – in this case, it delivered concrete results under pressure. This sets the stage for examining what makes NCD fundamentally different from traditional coding and why naming it properly matters.

Core Concepts of No Code Development

No Code Development (NCD) refers to a software development paradigm where the **developer does not manually write code** – at least not in the conventional sense. Instead, the developer interacts with one or more AI-powered systems (and often visual interfaces) using **natural language and other high-level inputs** to build an application. It is important to clarify that NCD is not about eliminating *logic* or *engineering thinking*; rather, it is about changing the interface of development from writing syntax to describing intent. The code still exists under the hood (generated by AI), but the human developer operates **one level higher**, concerned with *what* the software should do or look like, rather than *how to implement it in code*.

Some **core characteristics** define the NCD approach:

- **No Manual Coding:** The developer does not type out source code. All low-level implementation is handled by AI based on the developer's requests. This is the essence of "no code" in NCD – the human's role shifts from coder to *designer and orchestrator*. As a result, the **programming language becomes human language** (often English) ⁶. Recent observations echo this shift,

noting that with modern AI, “*the programming language is human*,” meaning anyone can instruct computers in plain English and get results that once required writing code ⁶ .

- **Iterative, Real-Time Feedback Loop:** NCD is highly interactive. The developer works in a tight loop of giving instructions (prompts) and receiving output (running code, UI previews, etc.), then refining further. This aligns closely with agile and rapid prototyping principles. In fact, the “*code first, refine later*” mindset is a core element of the approach ⁷ . One quickly gets a working version and then incrementally improves it by conversing with the AI. Because changes can be made via new prompts almost instantly, the workflow encourages continuous experimentation and refinement based on immediate feedback, rather than big upfront design.
- **Natural Language & High-Level Controls:** The primary “tool” for the developer is **natural language** – e.g., describing a feature: “*Create a dashboard with a chart of X vs Y, with these data points, and make it interactive*,” or “*The sidebar should be half as wide and in blue*.” The AI translates these into actual code changes. In many cases, this is complemented by **visual interfaces** or low-code style editors where the developer can drag-and-drop elements or adjust parameters, with the AI handling the plumbing. The key is that **intent is expressed declaratively** (what the developer wants) rather than imperatively (writing algorithms to do it).
- **AI Tooling as a Development Partner:** In NCD, AI is not just an autocomplete for code; it’s a **development partner** that can take high-level instructions and generate multi-layered code structures. Modern LLM-based tools (such as Replit’s Ghostwriter, Cursor, GitHub Copilot X, etc.) can create entire modules or UI components from a single prompt ² . The AI suggests implementations, fills in boilerplate, and even makes design suggestions in real-time ⁸ . The developer remains in the loop to guide and correct the AI (human-in-the-loop), but much of the heavy lifting is offloaded. This can dramatically speed up creating standard code patterns and free the human to focus on creative or complex decisions.
- **Building Real Software Products:** Perhaps most importantly, NCD is still **software development** in the full sense. Despite the lack of hand-written code, the outputs are real, working software artifacts – be it a web application, a data visualization, or a functional script. In the ThreatModCon case, for example, the outputs were fully functional UIs integrated with data and interactive behavior ⁵ . NCD is *not* merely conceptual or playing with mock-ups; the AI generates **executable code** under the hood. Thus, the *developer using NCD is still a developer* – they are making architecture decisions (albeit via guidance to the AI), composing features, and ultimately delivering a software solution. NCD practitioners build, orchestrate, and **develop real software products**; the difference is in *how* they issue instructions and who/what writes the code (an AI, not themselves).

It is worth comparing NCD to the earlier “*low-code*” or “*no-code*” *platforms* that existed before the AI boom. Traditional no-code platforms (such as website builders, app builders, workflow designers) allow people to create software via graphical interfaces and pre-built components, without coding. However, those platforms typically operate within **fixed, narrow domains** – you can only do what the platform’s blocks or templates support. Complex or unique requirements often hit a wall where coding becomes necessary, hence the rise of “low-code” (some coding allowed for extensions). **NCD, in contrast, offers far more flexibility.** Because an AI coding assistant can theoretically generate any code that a programmer could, an NCD approach is not limited to a small palette of components. If you can describe a feature in natural language (and it’s within the realm of what the AI + platform can do), you can attempt it. In practice there are still limitations – AI can make mistakes or might not fully understand very complex instructions – but the scope of possibility is much broader than with legacy no-code tools. This makes NCD especially attractive to technical users and professional developers: it’s not a toy sandbox, but rather **a new interface to the full power of coding**, one that can accelerate their work dramatically.

In summary, No Code Development aims to **capture the best of both worlds** – the approachability of no-code and the power of full-code development. It does so by shifting the developer's primary mode of interaction to natural language and high-level design, using AI to bridge the gap to actual code. The next sections will contrast this with traditional engineering practices and examine why *staying in this high-level "NCD zone"* can be so beneficial – and what challenges arise when one has to drop back down into manual coding.

Contrast with Traditional Engineering Practices

NCD represents not just a set of new tools, but a fundamentally different *mode of working* compared to traditional software engineering. In a conventional development workflow, a developer constantly oscillates between thinking about the problem conceptually and translating those ideas into code (with all the attendant concerns of syntax, debugging, configuration, etc.). With NCD, that translation step is largely outsourced to AI, allowing the human to remain at the conceptual level much longer. This difference creates a unique dynamic – and also introduces what we can call an **"air gap"** between the NCD process and traditional coding. Below, we outline key contrasts and the implications of context-switching between these modes:

- 1. Staying in the Creative Flow vs. Context-Switching to Code:** In NCD, the developer can remain *"in the zone"* of designing and refining the application behavior almost continuously. You express a requirement, see it materialize, and immediately iterate – all without stopping to manually code the solution. In traditional coding, even a small idea (e.g. *"make the sidebar half as wide"*) requires dropping out of creative flow, opening the code, finding the right file and line, editing CSS/HTML, then running the app to see if it looks right. Each such context switch is a friction point that can disrupt high-level thinking. Advocates of the NCD approach (formerly *vibe coding*) have noted this: *"The moment you look at code you're no longer vibing but into software engineering."* ³ In other words, the instant a developer has to dig into code, they've left the NCD mode and returned to the traditional mode where human attention is on minutiae. This **"air gap"** is essentially a break in flow. NCD seeks to minimize crossing that gap – the goal is to *not* look at the code if possible, and let the AI handle it. By doing so, the developer's focus stays on user experience, logic, and goals, rather than implementation details, leading to potentially more **fluid and uninterrupted productivity**.
- 2. Prompts and Dialogue vs. Specifications and Code Reviews:** Traditional development relies on formal specifications, tickets, design docs, and then writing code that meets those specs, followed by code reviews and testing. NCD, on the other hand, feels more like an ongoing **dialogue between the developer and the machine**. The "spec" is often implicit in a series of conversational prompts: you tell the AI what you want, it produces something, you refine or correct it with further instructions. The feedback loop is immediate, as opposed to writing code and later getting feedback from QA or users. However, this also means the *instructions (prompts) in NCD are not exact*. A prompt is more like a wish than a deterministic command ³ ; the AI interprets it with its own "understanding," which can be imperfect. This contrasts with code, where a specific line does a specific thing (assuming no bugs). Thus, **uncertainty is higher in each iteration of NCD**, and part of the developer's skill is learning to craft prompts and tests to coerce the AI into the desired outcome. In traditional engineering, precision comes from the developer; in NCD, the developer guides an inherently probabilistic agent. Both approaches ultimately require validation – testing the AI's output is as crucial as testing hand-written code.
- 3. Hallucination and Error Handling:** Because NCD relies on LLMs, we must contend with issues like AI hallucinations (plausible-sounding but incorrect outputs). As one commentator put it bluntly: *"All LLMs hallucinate 100% of the time. A lot of the times those hallucinations match some sort of reality."* ³ In a traditional setting, a compiler or runtime error is deterministic – something is either correct or it isn't, and a debug process ensues. In NCD, the AI might produce code that

runs but is subtly wrong (or inefficient, or insecure). This demands a new kind of vigilance from developers: they need to carefully review the *behavior* and results of AI-produced software, even if they aren't reading every line of code. In practice, an effective NCD developer will still do frequent reality checks – for example, if asking the AI to generate a data processing function, one should verify the output on known inputs, akin to writing tests. In this sense, *traditional engineering practices don't disappear; they get invoked at different points*. One might not do a manual code review upfront, but one might do a targeted review or test after the AI produces a module. There is a delicate balance: **too much diving into the AI's code** breaks the NCD flow (the air gap issue), but **too little oversight** risks errors slipping through.

4. **Speed of Development vs. Depth of Control:** A clear advantage of NCD is raw development speed. High-level features can be stood up in minutes, as seen in the ThreatModCon case. This is akin to having a very high-level programming language where one line does what would take 100 lines in a lower-level language. However, the trade-off is that the developer cedes some *fine-grained control*. In a traditional paradigm, an expert engineer might hand-craft an algorithm to be highly optimized or a UI component to be pixel-perfect. In NCD, you trust the AI's defaults to a larger extent; optimizing the performance or design beyond what the AI provides may require either iterative prompting or ultimately *dropping to code* if it's critical. This is why in mission-critical or performance-sensitive contexts, NCD might currently be used for initial prototyping, with a later phase of traditional optimization. Over time, as AI tools improve, the need to rewrite or optimize AI-generated code will lessen, but it remains an important consideration now.
5. **Emerging Best Practices vs. Established Discipline:** Software engineering as a field has decades of best practices, design patterns, and methodologies. By contrast, NCD (or vibe coding) is **still very new and evolving**. As Simon Wardley noted, *"The industry is still developing practices with the use of agents. There is no 'right way' at the moment, it's still emerging."*⁹ This means teams adopting NCD have to feel their way through new challenges: How to do version control for AI-generated code? How to ensure reproducibility of builds when an AI might produce slightly different outputs tomorrow? How to integrate NCD work into a larger project with a conventional codebase? These questions don't yet have fully standardized answers. In contrast, traditional engineering has well-trodden solutions (Git workflows, code reviews, CI/CD pipelines, etc.). Early adopters of NCD are in effect **pioneering new practices**. For example, one emerging practice is maintaining a log of prompts alongside the code (to document why the AI wrote it a certain way, akin to commit messages). Another is using NCD for front-end prototyping while keeping core backend logic written and reviewed by humans for now. These hybrid approaches attempt to get the best of both worlds while the NCD paradigm matures.

In summary, the contrast between No Code Development and traditional engineering comes down to a shift in the developer's locus of work: **from writing code to steering code generation**. The benefits in speed and creativity are clear, but they come with new challenges in quality control, maintainability, and process. One key lesson from practitioners is to avoid breaking the *"magic"* of NCD unless necessary – because shifting one's mental context fully back into code can derail the productivity gains. Cruz's experience resonates with this: as long as he could stay in the high-level mode (prompting out UIs and tweaking them in natural language), progress was rapid and smooth; had he been forced to stop and debug code for hours, it would have disrupted the entire flow of preparing the keynote. Therefore, an important part of mastering NCD is knowing **when to avoid the downward context switch** and how to structure one's workflow to **minimize the "air gap" interruptions**.

Nonetheless, there are times when bridging NCD and traditional coding is necessary (for example, integrating an AI-generated module into a larger hand-coded system, or troubleshooting a persistent bug the AI can't resolve on its own). In those moments, a developer effectively wears two hats – temporarily acting as a traditional engineer scrutinizing code, then returning to the NCD hat to continue the higher-level development. Teams and individuals will develop instincts for this over time, deciding

on the right balance. Next, we explore how developers – particularly seasoned ones – are uniquely positioned to excel in this new paradigm, and why rather than reducing the need for skilled engineers, NCD may actually **increase the demand for their expertise**.

Developer Empowerment via NCD – The Role of Experience and Domain Knowledge

One of the most surprising (and encouraging) aspects of the No Code Development revolution is how it **empowers developers – especially experienced ones** – to be more productive and creative than ever. It might sound ironic: if the AI is doing the coding, wouldn't a less-experienced person be just as effective as a 20-year veteran, since both are “not coding” in the classical sense? In practice, however, it turns out that **the best NCD practitioners tend to be those with a strong software engineering background**. Rather than making their skills obsolete, NCD amplifies the leverage of their expertise. Several factors explain why seasoned developers shine in this paradigm:

- **Domain Knowledge Shapes Better Prompts:** Crafting effective prompts for an AI is part art and part science. An experienced developer has a mental model of how software systems work – they understand architectures (MVC, client-server, etc.), know typical pitfalls, and have an intuition for what “good” solutions should look like. This knowledge helps them *ask for the right thing*. For example, a novice might prompt, “Make a website with a login page,” and accept whatever the AI gives. An expert might prompt, “Make a responsive single-page web app with OAuth2 login, a user profile dashboard, and role-based access control,” which is far more specific. The resulting output from the AI will likely be closer to the desired final product because the prompt encapsulated expert understanding. In essence, experienced devs can **encode their high-level design decisions into the prompt itself**, steering the AI more precisely. This often leads to needing *fewer iterations* to get things right, compared to a novice who might not specify important details up front.
- **Ability to Recognize and Correct AI Mistakes:** When an AI generates code or configurations, it may work superficially but hide subtle issues (security flaws, inefficiencies, edge-case bugs, etc.). A developer with years of experience can more readily *sense* when something might be off. They might say, “This output looks suspiciously simple for what I asked – did the AI handle edge case X?” and then test for it. Or they might notice the AI's code uses an outdated approach or vulnerable library, and prompt it to update. In Cruz's ThreatModCon development, for instance, his background in application security and engineering would let him spot if a graph visualization might not scale with more data, or if the AI's code to manipulate a knowledge graph could be optimized. Thus, **veterans act as editors and safety nets** for the AI's work. They know when to trust the AI and when to verify carefully.
- **Orchestration and Integration Skills:** Building software is not just writing one piece of code; it's assembling many pieces into a coherent whole. Experienced engineers are adept at this orchestration. In an NCD setting, this might mean breaking down a complex app into segments that can each be prompt-developed, then combining them. For example, an expert NCD developer might say: “First, I'll have the AI generate a backend API for my idea. Next, a frontend UI. Then, I'll use a natural language prompt to have it connect the two.” They effectively **architect the solution at a high level**, then use AI to fill in the implementation of each part. This mirrors how senior devs lead projects, except now their “team” includes AI assistants. An inexperienced person might struggle to divide the problem or to integrate components, even if AI builds each piece. So, we see that **system design aptitude** is still crucial – NCD doesn't remove the need for thinking in terms of modules, data flow, and user journey.
- **Focus on Higher-Order Problems:** By freeing developers from writing boilerplate and routine code, NCD allows them to tackle more complex or creative tasks in the same amount of time. An

experienced developer armed with NCD can solve higher-order problems that were previously out of scope due to time or resource constraints. This is empowering: they can try out bold ideas quickly, or address user needs on the fly. In the keynote example, Cruz could spontaneously implement audience suggestions or new threat model visualizations during practice sessions, because adding a feature was as simple as prompting it. The veteran developer's broad skillset means they have more of these ideas to try in the first place (they recall many patterns and use-cases from their career). Now, with NCD, they can *actually implement those ideas immediately* rather than shelving them due to limited time. This leads to an explosion of productivity and innovation – essentially, **experienced devs become force multipliers**, achieving in hours what used to take days, and exploring many more options along the way.

- **NCD Does Not Eliminate Developers – It Leverages Them:** It's critical to counter the misconception that because the AI writes code, human developers will not be needed. On the contrary, real-world adoption of NCD has shown that it tends to **increase the demand for skilled developers** within organizations. As Cruz noted, the area where these AI coding techniques add value (like custom UIs on top of solid backends) *"will need more, not less devs."* ¹ Why? Because once you enable software to be created faster, the scope of what people want to build also grows. Business units realize they can ask for more tailored tools; product ideas that were on the backlog can be attempted sooner. The result is *more projects*, not fewer. And while AI can handle the grunt work, human developers are still required to oversee projects, refine AI output, integrate systems, and ensure quality. One LinkedIn commentary succinctly observed that due to the explosion of demand caused by vibe coding/NCD, *"most organizations will end up needing more software engineers."* ⁹ Rather than replacing programmers, NCD **elevates their role**: developers become the conductors of an AI orchestra. Their expertise is essential to keep the music playing well. Non-technical users, while empowered by easier interfaces, typically won't be designing complex systems solo – they will collaborate with developers who understand both the domain and the AI tools.

- **The Learning Curve and Red Queen Effect:** It should be noted that becoming proficient in NCD is its own learning journey. Experienced developers must augment their skills with prompt engineering and AI tool familiarity. As one expert quipped, you should *"spend a few hundred hours vibe coding when you never look at code, then a few hundred more [with] software engineering + AI (i.e. review the code that AI writes)"* ¹⁰. The message here is that developers should practice both the pure no-code approach and the hybrid approach of checking AI code, to truly master the new workflow. Those who invest the time to learn NCD will outpace those who do not – the so-called **Red Queen effect** (you have to keep running just to stay in place, given how fast the field moves). For seasoned developers, this is an opportunity: their background gives them a head start, and with some dedicated practice in NCD, they can become the go-to experts of this new paradigm. We are already seeing early adopters (many of them experienced engineers) share "best practices" for vibe coding safely, tips on how to avoid frustration, and frameworks for collaborating with AI in development. This growing body of knowledge will further empower developers as it solidifies.

In conclusion, No Code Development doesn't diminish the importance of developer skill – it **rewards it in new ways**. The AI may handle the typing, but it's the human's insight that drives success. Experienced developers find that NCD lets them apply their knowledge at a more abstract level, multiplying their impact. There is a rich irony in this: a practice that on the surface looks like it could make programming trivial actually ends up highlighting just how valuable a programmer's experience really is. In NCD, *the wizardry isn't in writing code by hand, but in knowing what to build and how to guide the machine to build it*. This is why giving this paradigm a proper name and conceptual clarity matters – so we can properly position developers (especially senior ones) to embrace it and organizations to

recognize its value. In the next section, we'll discuss strategic implications for teams and tooling, and why we advocate for "No Code Development" as a clear conceptual identity distinct from earlier notions of low-code or no-code.

Strategic Implications for Teams and Tooling

Adopting No Code Development at an organizational level has far-reaching implications. It's not just a technical adjustment, but a cultural and process shift. Here, we outline strategic considerations for teams, tooling, and the overall development lifecycle, especially in comparison to traditional low-code platforms. We also underscore why using the term **NCD** (No Code Development) helps in framing these changes for both technical and executive stakeholders.

1. Integrating NCD into Team Workflows: Engineering teams can greatly benefit from NCD, but it requires rethinking how projects are executed. One approach is to introduce NCD in the **prototyping and design phase**. For instance, instead of spending weeks building a proof-of-concept, a small team could "vibe code" a prototype in a few days, get stakeholder feedback, and iterate rapidly. This was exemplified by the ThreatModCon case – what traditionally might have been a long development of demo tools was compressed into interactive prototyping sessions. Teams should establish guidelines on *when* NCD is appropriate (e.g. early-stage exploration, internal tooling, UI mockups, etc.) and when to transition to traditional engineering (e.g. for hardened production code, performance-critical modules, or long-term maintenance). Over time, the line will move as NCD tools become more robust, but initially this hybrid model can yield quick wins. It's also wise to incorporate **peer review of AI outputs**: for example, after using NCD to generate a component, have another developer skim the AI's code suggestions or at least validate the functionality. This marries the speed of NCD with the safety nets of traditional dev processes.

2. Training and Skill Development: Teams should invest in training developers to become fluent in this new paradigm. This includes learning how to write effective prompts, understanding the capabilities and limits of the chosen AI coding tools, and knowing how to fix things when the AI inevitably does something unexpected. There is also a mindset shift: developers need to become comfortable with a less deterministic process. Mentoring programs or knowledge-sharing sessions can help; for instance, early adopters of NCD within the team can demonstrate their workflow to others (perhaps showing how they built a feature via prompts and how they handled any AI missteps). Encouraging a blameless, experimental culture is key – NCD works best when developers aren't afraid to try bold prompts or throw away AI-generated code and regenerate if needed. Leadership can play a role by **celebrating productivity gains** (e.g., "We delivered that feature 3x faster using NCD!") and framing errors as learning opportunities rather than failures. By formalizing NCD as an accepted practice, experienced devs will feel empowered to use their judgment on when to employ it versus hand-coding. This avoids the stigma that might come if it remained "vibe coding" (which some might dismiss as a toy); calling it No Code Development and treating it as a valued skill helps legitimize it.

3. Tooling and Infrastructure Support: Just as DevOps rose to prominence to automate and support traditional code pipelines, we will see new tooling to support NCD workflows. Organizations should evaluate what platforms or AI assistants align with their security and quality needs. Some may opt for self-hosted LLM solutions (for privacy/IP reasons) or specific IDEs that integrate AI deeply. Continuous Integration (CI) processes might need tweaks – for example, if code is AI-generated, ensuring that the generation can be reproduced later is important (to avoid a scenario where an AI output can't be regenerated the same way, complicating future maintenance). One strategy is to **store not just the code, but also the prompts or conversation logs** that led to the code ¹⁰. This acts as a form of documentation. Tool vendors are already exploring features like "prompt version control" and "AI action

history” for this purpose. Additionally, testing and monitoring become even more crucial. Teams should bolster their automated testing suites – treat AI-written code as you would an opaque third-party library: trust but verify. If an AI generates a piece of logic, write tests for its expected behavior. In production, use monitoring to catch anomalies that might arise from unanticipated AI code behavior. Essentially, classic software engineering quality practices remain necessary, but they wrap around the NCD process, catching issues that might slip through from the AI generation phase.

4. Security and Governance: For organizations, especially in sensitive industries, the adoption of NCD must be accompanied by governance policies. AI-generated code can introduce vulnerabilities if not checked (for instance, using an outdated library version with known CVEs, or writing insecure SQL queries due to an incomplete prompt). Security teams should develop guidelines for NCD usage, perhaps requiring security scans of AI-produced code or disallowing NCD for certain high-risk components unless the AI models are security-vetted. Interestingly, NCD can also be turned to advantage in security – developers could ask an AI to *harden* or *review* its own code for vulnerabilities, effectively prompting it to be a security assistant. But this is an evolving area. Governance also extends to intellectual property: using cloud-based AI coders might raise IP questions (who owns the generated code, does it introduce any licensed content?). By officially embracing NCD as a concept, organizations can have these discussions and set policies, rather than pretending it’s not happening. The term “No Code Development” gives a concrete handle to discuss these issues at the management level, whereas “vibe coding” might not be taken seriously in a board meeting. Thus, adopting the NCD terminology can facilitate the necessary **executive buy-in and oversight**.

5. Differentiating NCD from Traditional Low-Code/No-Code Platforms: It’s important for technical leaders to recognize – and communicate – that NCD is not the same as the old wave of no-code platforms. We use the phrase “*No Code Development*” deliberately to carve out a distinct identity. Traditional no-code tools (think Wix, Squarespace, Microsoft PowerApps, etc.) were generally aimed at non-developers and often sidelined by software teams because of their limitations. In contrast, NCD as described here is **developer-centric** and far more flexible. It harnesses *general-purpose AI coding assistants* rather than a fixed palette of widgets. For example, a low-code platform might let you build a form-based business app quickly, but try to do something truly custom and you hit a wall. With NCD, if you can describe the custom feature, you can attempt to build it – the AI might write entirely new code to fulfill your request, not just stitch together existing modules. This is a game-changer for technical teams. They can build internal tools or bespoke features without the overhead of full coding, yet without the constraints of a closed platform. When communicating to stakeholders, it’s useful to position NCD as “**the next evolution of development**”: just as assembly gave way to high-level languages, and high-level languages to frameworks, now we move from frameworks to **intent-driven development**. Using the right terminology helps here. Saying “we’re using no-code development” can initially raise eyebrows (“Are the developers being replaced by business users with drag-and-drop tools?”), so it must be clarified that **NCD is a technique for developers, by developers, leveraging AI**. It exists in a different category than the citizen-developer no-code tools, though they share the surface similarity of “not typing code”. In fact, one could argue for a new taxonomy: perhaps call those older platforms “*pre-defined no-code*” and NCD as “*AI-assisted no-code*.” For now, simply using *No Code Development (NCD)* as the term in technical roadmaps and team charters will signal a forward-looking, technical approach rather than a regression to simplistic tools.

6. Team Roles and Collaboration: As NCD becomes more prevalent, teams might also see shifts in roles. For example, the role of a “**Prompt Engineer**” might emerge distinctly – someone who specializes in crafting and optimizing prompts for the AI to get the best results (though in many cases this will just be a skill that all developers cultivate). We might also see more cross-functional collaboration: a domain expert or designer could directly contribute by describing desired features in natural language which the developer+AI then turn into reality. In essence, NCD can make development more inclusive, as

natural language is a common denominator between coders and non-coders. Strategically, managers should encourage this cross-pollination. If a UX designer can “sketch” a UI by writing a paragraph of description to the AI with a developer refining it, that removes barriers between teams. This was hinted at in Cruz’s note that *vibe coding* adds value especially in “*creating custom UIs for users on top of solid architecture*”¹ – a space where understanding user needs (often a designer’s domain) and understanding tech (developer’s domain) intersect. NCD tooling could allow a more collaborative canvas where, say, a product manager writes “I want a report page that shows X, Y, Z” and the developer uses that as the direct input to build with AI. Embracing NCD means nurturing this kind of fluid collaboration.

In sum, the strategic takeaway is that **No Code Development should be approached proactively, not reactively**. Companies that explicitly adopt NCD, define what it means for them, and educate their teams will harness its advantages more effectively than those that let it creep in ad-hoc (for instance, individual developers quietly using ChatGPT to code faster without any team knowledge or support). By naming and embracing the paradigm, leadership can ensure that *practices, tools, and governance evolve in tandem*. This is why we advocate for giving it a clear name like NCD – it’s a banner under which to rally efforts, invest in tooling, and communicate value.

Conclusion and Future Directions

Software development is undergoing one of the most significant paradigm shifts in its history. The rise of AI-driven development – exemplified by what we here call **No Code Development (NCD)** – is transforming how we create software, who can create software, and how quickly ideas can be turned into reality. In this paper, we argued that the term “*vibe coding*”, while catchy and indicative of the trend, does not do justice to this transformation. Instead, “**No Code Development**” provides a clearer, more professional, and accurate descriptor for the paradigm wherein developers build real applications through natural language and iterative AI assistance, without manually typing code.

Through the lens of Dinis Cruz’s ThreatModCon 2025 keynote case study, we saw tangible evidence of NCD’s power. Complex visual tools were developed in a flash, multiple iterations were done in the eleventh hour, and the outcome was successful – all of this would have been nearly unthinkable with manual coding under similar time constraints. The case study highlighted how *workflow (iterative and feedback-driven)* and *interface (natural language and UI)* in NCD fundamentally differ from traditional coding, enabling a level of agility and responsiveness that conventional methods struggle to match.

We examined why the **moniker “vibe coding” is insufficient**: it’s informal, potentially misleading, and doesn’t convey the real work being done. In contrast, “No Code Development” directly communicates the key attribute – no code writing by the human – while including the word “Development,” affirming that this is a genuine software development practice, not a gimmick. For technical and executive audiences, this distinction is crucial. As organizations consider adopting these techniques, having a term like NCD helps frame policies, job roles, and expectations. It distances the conversation from any notion that this is just some passing fad where people “code by vibes,” and instead positions it as an **evolution of how software can be built**.

Looking ahead, we can foresee several **future directions** and needed efforts in the realm of NCD:

- **Formalization of NCD Best Practices:** Just as agile development, DevOps, and other movements eventually produced playbooks and frameworks, NCD will benefit from more formal best practices. This could involve standardized ways to document prompt-driven development (e.g. incorporating prompt logs into design documentation), methodologies for ensuring quality (like “Test-Driven Prompting” where you write the test criteria before generating the code), and

design patterns specific to AI-generated code. We are likely to see community-driven guides and perhaps even certifications around NCD competence, which will further cement the legitimacy of the approach.

- **Tool Ecosystem Growth:** The current set of AI coding assistants will likely evolve into more integrated **NCD platforms**. These might include features such as: project-wide AI context (so the AI understands your whole application when generating code, not just the current prompt), seamless fallback to human coding when needed (with AI able to explain its code to a human developer in comments, for example), and advanced debugging aids (imagine an AI that not only writes code, but when a test fails, can analyze the failing scenario and adjust its output accordingly). We might also see specialized NCD tools for different domains (e.g. an AI tuned for front-end development vs. one for data engineering), which could be chosen as appropriate. An exciting possibility is AI agents that work in tandem – one generates code, another reviews it – functioning like an automated pair programming team under human supervision.
- **Bridging the Gap to Traditional Engineering:** In the near term, one of the challenges (as discussed under the “air gap”) is transitioning AI-generated prototypes into maintainable, production-quality code bases. Future advancements might narrow this gap. For instance, AI systems might be trained to generate not just code, but also accompanying documentation, test cases, and even formal specifications (ensuring that what’s generated is easier to maintain and verify). The concept of *“round-trip” development could emerge: a human writes some code, an AI improves or refactors it, another human tweaks it, and so on – blurring the line between NCD and traditional coding until it’s all one continuum of development. In such a future, the term “No Code Development” might evolve again – perhaps simply becoming “AI-augmented development”* as virtually all coding involves AI. But for now, we use NCD to clearly delineate the practice as it stands.
- **Workforce and Culture Evolution:** As NCD gains traction, we should prepare for shifts in team composition and culture. We touched on the potential emergence of roles like prompt engineers or more fluid collaboration between technical and non-technical team members. In the long run, it’s conceivable that what we consider a “developer” broadens. The next generation of developers might be those who grew up with AI collaborators and thus approach problem-solving differently – more experimental, less inhibited by syntax or fear of making mistakes (since the AI can always regenerate a fresh attempt). Organizations that embrace NCD early will be attractive to this emerging talent pool, as they will have an environment that aligns with the new way of creating. Culturally, emphasizing creativity, adaptability, and continuous learning will be more important than ever. NCD reduces the premium on memorizing API calls or mastering a single framework, and increases the premium on understanding systems conceptually and having a breadth of knowledge to guide AI. Team cultures should reward those who can think holistically and work iteratively with AI, rather than just cranking out code manually.
- **NCD for Technical Users vs Citizen Developers:** We also pointed out the need to distinguish NCD from the “no-code for everyone” narrative. In the future, these two threads might converge or remain parallel. It’s possible that as AI interfaces become extremely user-friendly, more non-programmers will successfully create software through conversation (fulfilling some of the promise of old no-code tools but with far greater power). However, our focus in this paper has been NCD for **technical users** – people with engineering mindset and knowledge leveraging it to go faster. We foresee that these technical practitioners will define the patterns that later allow broader audiences to safely use NCD tools. In essence, experienced devs will pave the way, figuring out how to harness AI development reliably. Once that knowledge is distilled into robust tools, then citizen developers can partake more without stepping on landmines. So, the immediate future should see *technical NCD* clearly delineated and nurtured (again, using a term like NCD helps here), and the longer-term future might see a democratization once the kinks are worked out. It’s important that we do **not conflate these in the present**; failing to differentiate could either scare engineers away (“this is just for non-coders, not serious”) or conversely set

non-coders up for failure (“just vibe code an app, what could go wrong?”) if they try without guardrails.

- **Community and Knowledge Sharing:** Lastly, a future direction is simply the growth of an NCD community of practice. Already, developers are sharing tips in blog posts, LinkedIn articles, and conferences. Dinis Cruz’s own journey – sharing slides and writing about his experiences – is part of building this collective knowledge ⁵ ¹. We anticipate dedicated forums, maybe even conferences, on No Code Development where people discuss prompt crafting, showcase AI-built projects, and address challenges (like how to reduce hallucinations, or how to persuade management to trust AI-generated code). This community aspect will be vital to refine the paradigm. Just as open source communities accelerated software development in past decades, open sharing of NCD techniques will propel this field forward quickly. It’s notable that this white paper itself is a product of that ethos: blending insights from direct experience with public discussions and documentation to chart a path forward.

In closing, embracing **No Code Development** is about recognizing that the essence of software creation is moving to a higher plane. It’s not that code ceases to exist – it’s that writing it becomes a job we increasingly delegate to machines, so humans can focus on creativity, design, and problem-solving at the conceptual level. By choosing terminology that reflects that reality, we help everyone – developers, executives, and stakeholders – understand what’s really happening. “*Vibe coding*” was a colorful introduction to the idea, but **NCD is the term that can carry it into everyday practice**. Dinis Cruz’s experiences and advocacy exemplify how an expert developer can lead this change: demonstrating the results, highlighting the pitfalls, and pushing for a better language to discuss it. We hope this paper has articulated why No Code Development deserves its place in the lexicon of modern software engineering and inspired others to explore this paradigm deeply. The vibes of coding may be good, but the future of development is one where we **build more by coding less**, and we should name it boldly as such.

¹ ³ ⁹ Vibe coding: a bad name for a good idea | Dinis Cruz posted on the topic | LinkedIn
https://www.linkedin.com/posts/diniscruz_spot-on-analysis-on-vibe-coding-the-area-activity-7329090329624653825-kT5h

² ⁷ ⁸ What is Vibe Coding? | IBM
<https://www.ibm.com/think/topics/vibe-coding>

⁴ Not all AI-assisted programming is vibe coding (but vibe coding rocks)
<https://simonwillison.net/2025/Mar/19/vibe-coding/>

⁵ ThreatModCon 2025 Barcelona | Dinis Cruz | 12 comments
https://www.linkedin.com/posts/diniscruz_threatmodcon-2025-barcelona-activity-7334649014136582145-J6et?trk=public_post_comment-text

⁶ Portuguese as a Programming Language in the AI Era - Dinis Cruz - Documents and Research
<https://docs.diniscruz.ai/2025/02/11/portuguese-as-a-programming-language-in-the-AI-Era.html>

¹⁰ spot on | Dinis Cruz
https://www.linkedin.com/posts/diniscruz_spot-on-activity-7330551958170173442-HsKe